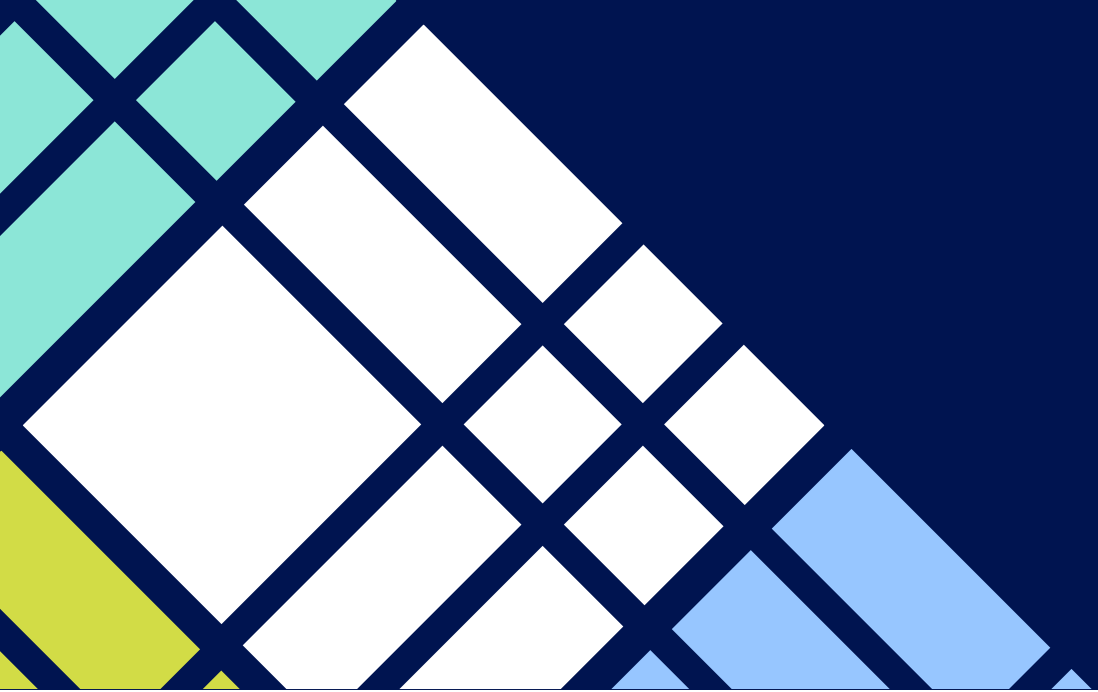


Dresden University of Technology

Dresden Database Research Group

Week 8: JSON in SQL

Practical Course: SQL, Summer Semester 2026

Dresden University of Technology

Dresden Database Research Group

3

Title

Subtitle

Dresden University of Technology

Dresden Database Research Group

Title

Subtitle

Subtitle

Headline

Dresden Database Research Group

Text

- Other text 1
- More text 1

Text 2

- Other text 2
- More text 2

Headline

Dresden Database Research Group

Text

- Content

Text

- Content

Headline

Dresden Database Research Group

Text

- a

Text

- b

Text

- b

Headline

Introduction to JSON

Dresden Database Research Group

JSON building blocks

- Object: unordered key-value pairs
  - *e.g.,* `{"name": "Daniel Krueger", "salary": 4867}`
- Array: ordered list of values
  - *e.g.,* `["Python", "SQL", "C++"]`
- Values can be: string, number, `null`, object, array

```
{
  "name": "Daniel Krueger",
  "salary": 4867,
  "city": "Dresden",
  "performance": {
    "rating": 4, "projects": 7, "reviews": 12
  },
  "skills": ["Python", "SQL", "C++"]
}
```

Idea

- Relational tables need fixed schema i.e. every row has the same columns
- Some data is semi-structured e.g., some employees have many skills, some don't
- JSON allows to store a flexible document inside a single column

JSON support is varying by database vendors!

JSON in PostgreSQL

How is JSON stored?

- As a column value: One JSON document per row
- PostgreSQL offers two types: `json` and `jsonb`

Performance

- `jsonb` reads are faster
  - no repeated parsing + supports indexes (e.g., GIN)
- `json` writes are faster
  - stores original text as-is → less processing during insert/update

→ Use `jsonb` by default for (focus of this lecture)

id	profile
1	{"name":"Daniel Krueger", "salary": 4867, ...}
2	{"name":“Sofia Weber", "salary": 6074, ...}
3	{"name":“Lukas Brandt", "salary": 3854, ...}
4	{"name":“Amelie Fischer", "salary": 5291, ...}

Tab. 1: Employees

	`json`	`jsonb`
Storage	Raw text	Parsed binary structure
Access	Re-parses text every time	Reads pre-parsed structure
Indexable	No	Yes
Operators	Limited	Full set

Tab. 2: `json` vs. `jsonb`

Access Operators (1)

Dresden Database Research Group

Simple Access

- Accessing individual JSON fields
  - `->` returns `jsonb` (keeps JSON typing)
  - `->>` returns `text` (plain value)

```
SELECT
  profile ->  'city' AS arrow1 -- “Dresden“ (jsonb)
  profile ->> 'city' AS arrow2 --  Dresden  (text)
FROM employees WHERE id = 1;
```

id	profile
1	{"name":"Daniel Krueger", "salary": 4867, ...}
2	{"name":“Sofia Weber", "salary": 6074, ...}
3	{"name":“Lukas Brandt", "salary": 3854, ...}
4	{"name":“Amelie Fischer", "salary": 5291, ...}

Usage in `WHERE`

- Typically “->>” for string comparisons

```
SELECT
  profile ->> 'name'   AS name
  profile ->> 'salary' AS salary
FROM employees
WHERE profile ->> 'name' = 'Amelie Fischer';
              ->>  text  = text
              ->   jsonb = text
```

Casting values

- Use `::int` to cast text to integers
- Needed for numeric `ORDER BY`, `AVG`, `>`, ...

```
SELECT SUM((profile ->> 'salary')::int)
FROM employees;
```

Access Operators (2)

Dresden Database Research Group

Path Access

- For nested access, two styles
  - Chain: `profile -> 'performance' ->> reviews`
  - Path: `profile #>> '{performance,reviews}'`
- Are more concise for deep nesting

```
{
  "name": "Daniel Krueger",
  "salary": 4867,
  "city": "Dresden",
  "performance": {
    "rating": 4, "projects": 7, "reviews": 12
  },
  "skills": ["Python", "SQL", "C++"]
}
```

Summary

- The 4 access operators

	Returns `jsonb`	returns `text`
by key/index	`->`	`->>`
by path	`#>`	`#>>`

Existence Operators (1)

Dresden Database Research Group

Containment @>

- “Does the left side contain the right side”
  - On an object: e.g. all employees living in Berlin

```
SELECT profile -> 'name' AS name
FROM employees
WHERE profile @> '{"city": "Berlin"}';
```

  - On an array, e.g. all employees who speak Python AND SQL

```
SELECT profile -> 'name' AS name
FROM employees
WHERE profile -> 'skills’ @> '["Python", "SQL"]';
```

id	profile
1	{"name":"Daniel Krueger", "city": “Dresden", ...}
2	{"name":“Sofia Weber", "city": “Berlin", ...}
3	{"name":“Lukas Brandt", "city": “Munich", ...}
4	{"name":“Amelie Fischer", "city": “Berlin", ...}

Unnesting functions (1)

Dresden Database Research Group

Motivation

- So far: Scalar field access e.g., `profile -> 'name'`
- What about accessing arrays?
  - *e.g., counting the skills per employee* (`"skills": ["Python", "SQL", "C++"]`)
- … or objects with dynamic keys?
  - *e.g., listing performance per employee* (`performance": {"rating": 4, "projects": 7, "reviews": 12}`)

→ Problem: A fixed-key `->>` can't iterate over unknown/variable contents

→ Solution: Turn multi-element JSON structures into rows

Unnesting functions (2)

Dresden Database Research Group

`jsonb_array_elements`

- Used to unpack over arrays
- Returns one row per array element
  - *e.g.,* `SELECT jsonb_array_elements('["Python", "SQL", "C++"]'::jsonb) AS skill;`

skill
“Python”
“SQL”
“C++”

`jsonb_each`

- Used to unpack JSON objects
- Returns one row per key-value pair as two columns
  - *e.g.,* `SELECT * FROM jsonb_each('{"rating": 4, "projects": 7, "reviews": 12}'::jsonb);`

key	value
„rating“	4
„projects“	7
„reviews”	12

→ Both are set-returning functions

→ How to expand to expand e.g., skills per employee?

Unnesting functions (3)

Dresden Database Research Group

Unnest a column for every row (1)

- Example: Unnest “skills” per employee

```
SELECT
  e.profile ->> 'name' AS name,
  _skill #>> '{}'      AS skill
FROM employees e,
  LATERAL jsonb_array_elements(e.profile -> 'skills') AS _skill;
```

id	profile
1	{"skills": ["Python", "SQL", "C++"], …}
2	{"skills": [“Java”,], …}
3	{"skills": [], …}

- Referencing another table’s column from a function in `FROM` requires `LATERAL`

Result

name	skill
Daniel Krueger	Python
Daniel Krueger	SQL
Daniel Krueger	C++
Sofia Weber	Java

Unnesting functions (4)

Dresden Database Research Group

Unnest a column for every row (2)

- Example: Unnest “performance” per employee

```
SELECT
  e.profile ->> 'name' AS name,
  kv.key               AS discipline,
  (kv.value)::int      AS cnt
FROM employees e,
  LATERAL jsonb_each(e.profile -> 'performance') AS kv;
```

id	profile
1	{"performance": {"rating": 4, "projects": 7, "reviews": 12}, …}
2	{"performance": {"evaluations": 6}, …}
3	{"performance": {}, …}

Result

name	discipline	cnt
Daniel Krueger	„rating“	4
Daniel Krueger	„projects“	7
Daniel Krueger	„reviews“	12
Sofia Weber	„evaluations“	6

Constructing JSON (1)

Dresden Database Research Group

jsonb_build_object

- Builds an JSON object from columns
- Nest calls to build nested objects
- Syntax: jsonb_build_object('key1', val1, ‘key2', val2, ...)
- Example:

```
SELECT jsonb_build_object(
    'name', e.name,
    'perf', jsonb_build_object(
        'rating',   e.rating,
        'projects', e.projects)) AS doc
FROM employees e WHERE id = 1;
```

- Result:

```
{"name": "Daniel Krueger", "perf": {"rating": 7, "projects": 4}}
```

employees

name	rating	projects
Daniel Krueger	7	4
Sofia Weber	8	6

Constructing JSON (2)

Dresden Database Research Group

jsonb_agg

- Aggregates many rows into one JSON array
- Supports `DISTINCT`, `ORDER BY` and `FILTER` (`WHERE` …) inside
- Example: “Get employees per city”

```
SELECT profile ->> 'city' AS city,
       jsonb_agg(
             profile ->> 'name'
             ORDER BY profile ->> 'name'
         ) AS members
FROM employees
GROUP BY profile ->> 'city';
```

- Result

city	members
Berlin	[”Amelie Fischer”, ”Sofie Weber”]
Cologne	[”Lukas Brandt”]
Dresden	[”Daniel Krueger”]